claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\workflows\wf_8d68d336-288\agent-a18af2691a7fb0b36.jsonl`
- SHA-256: `262c27ba3d99519a57aa937b1508857838c30259b9d03d4a02755e09432e5d9a`
- Source modified: `2026-07-03T19:54:13+00:00`
- Imported at: `2026-07-08T16:00:15+00:00`
- project: `wf_8d68d336-288`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-03T19:52:13.893Z)

Repo (my in-progress optimization branch): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/fix-worktree
Key files to Read:
- backend/pipeline/detectors/wasb_infer.py ? the inference loop (DataLoader ? detector.run_tensor ? tracker). Contains the streaming SequentialFrameStore, the _PrefetchIter I just added, build_windows, and the tracker re-run loop.
- backend/pipeline/detectors/native_wasb_runner.py ? builds the wasb_infer.py argv on the GPU box (native runner). Has device/stream_video/batch_size/prefetch_batches config.
- backend/config/models.py (WasbIndexConfig) and backend/config/presets.py (cloud-serving preset).
- deploy/cloudrun/Dockerfile ? the WASB conda env is torch 1.11.0+cu113 (py3.8), a SEPARATE env from the app's torch. The detector shells out to it as a subprocess.
The WASB model itself lives at /opt/models/WASB-SBDT (nttcom/WASB-SBDT) ? you can't read it, but reason from how wasb_infer.py calls build_detector(cfg) / detector.run_tensor(imgs, trans) and build_tracker(cfg).

MEASURED BASELINE (first real L4 cloud run, 2026-07-03): a ~7.5-min 60fps badminton clip took Phase-2 (WASB detection) 727.9s (~0.6x realtime); GPU utilization sat <1% the whole time. Hardware: NVIDIA L4 (Ada sm_89, 23GB), 8 vCPU, 32GB. Streaming mode forces DataLoader num_workers=0 (single-process), so CPU batch assembly (cv2 decode + PIL + ToTensor) serializes with GPU inference.

CRITICAL CONSTRAINT ? the D5/D16 byte-parity guarantee: the repo REQUIRES that the rally/window OUTPUT (the candidate windows and trajectory CSV) stay bit-identical / deterministic across runs and across the disk-frames vs streaming paths. This is asserted by a real parity test. An optimization that changes detector NUMERICS (FP16, TF32, cudnn.benchmark algo selection, NVDEC vs CPU decode) risks changing the output and breaking that guarantee. Batch size and prefetch do NOT change per-frame numerics (same forward, just grouped/overlapped) so they are parity-safe. Already implemented on this branch: indexing.wasb.batch_size + prefetch_batches knobs, the cloud-serving preset opts into batch 64 + prefetch 4.

Here are 0 analyzed + parity-verified optimization levers for the WASB-on-L4 inference path:

[]

Produce the definitive optimization plan as markdown. Requirements:
1. An ORDERED action list: highest (expected_win × parity_safety × low-effort) first. Separate "ship-as-default (parity-safe)" from "ship-as-opt-in (accepts new baseline)" from "measure-first" from "reject".
2. For the torch cu113?cu12x bump specifically: state the concrete recommended version and whether it is the single biggest safe win (the <1% util + JIT story suggests it might be).

3. Call out the ONE lever most likely to be the true bottleneck given "CPU-feed-bound, GPU <1%" ? is it really the GPU kernel at all, or the CPU decode/transform feed? Batch size won't help a feed-bound pipeline much; say so if true.
4. A concrete measurement protocol: what to A/B on the same video/instance, what metric (Phase-2 wall + GPU duty cycle), and the parity check that must pass before each default-on change ships.
5. Flag any proposed change that would break the byte-parity test and thus must be an explicit opt-in, not a silent default.
Be decisive and specific. This becomes the optimization section of the DEPLOY_REPORT and the plan for the next PRs.

2. assistant (2026-07-03T19:52:16.688Z)

I'll analyze the actual code before producing the plan. Let me read the key files.

3. user (2026-07-03T19:52:19.539Z)

```
1       """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3       WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4       own building blocks (ImageDataset transform + detector + online tracker) to run on
5       any folder of extracted frames and emit a trajectory CSV in the exact shape the
6       indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8           Frame,Visibility,X,Y
9
10      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14      Resumable across reboots
15      ------------------------
16      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17      the slow, expensive part; the CPU tracker pass that turns detections into a
18      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19      resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
30
31      The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32      to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33      weights, sport, window size, or frame count change.
34
35      Usage (in WSL, from ~/models/WASB-SBDT/src):
36          python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37              --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
38      """
39
40      import argparse
41      import csv
42      import glob
43      import json
44      import logging
45      import os
46      import os.path as osp
47      from collections import defaultdict
```

```
    from contextlib import contextmanager

    logger = logging.getLogger(__name__)

    CACHE_VERSION = 1
    _DET_FILE = "det_raw.jsonl"
    _MANIFEST_FILE = "manifest.json"


    def _build_cfg(weights: str, sport: str, device: str = "cuda"):
        """Compose the WASB eval config via Hydra, overriding model/weights/device.

        ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
preserved),
        ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
``runner.gpus=[0]``;
        cpu/mps run with no GPU list so the detector places tensors on ``device``.
        """
        from hydra import compose, initialize

        gpus = "[0]" if device == "cuda" else "[]"  # single GPU on cuda; none for cpu/mps
        with initialize(config_path="configs", version_base=None):
            cfg = compose(
                config_name="eval",
                overrides=[
                    f"dataset={sport}",
                    "model=wasb",
                    f"detector.model_path={weights}",
                    f"runner.device={device}",
                    f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to
this box
                ],
            )
        return cfg


    def _frame_id(path: str) -> int:
        """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
        stem = osp.splitext(osp.basename(path))[0]
        digits = "".join(ch for ch in stem if ch.isdigit())
        return int(digits) if digits else -1


    # ---------------------------------------------------------------------- #
    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
    # ---------------------------------------------------------------------- #
    def _cache_paths(cache_dir: str):
        return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


    def default_cache_dir(out_csv: str) -> str:
        """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
        d = osp.dirname(osp.abspath(out_csv))
        stem = osp.splitext(osp.basename(out_csv))[0]
        return osp.join(d, stem + "_wasbcache")


    def _atomic_write_text(path: str, text: str) -> None:
        """Write then rename so a crash mid-write can't leave a half-written file."""
        tmp = path + ".tmp"
        with open(tmp, "w") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
```

```
        os.replace(tmp, path)


    def _det_plain(det) -> list:
        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
        xy = det["xy"]
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


    def _dets_to_tracker(plain_list):
        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
        the tracker does vector math / np.linalg.norm on it)."""
        import numpy as np

        return [
            {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
            for p in plain_list
        ]


    def write_manifest(cache_dir: str, manifest: dict) -> None:
        _, mpath = _cache_paths(cache_dir)
        _atomic_write_text(mpath, json.dumps(manifest, indent=2))


    def read_manifest(cache_dir: str):
        _, mpath = _cache_paths(cache_dir)
        if not osp.exists(mpath):
            return None
        try:
            with open(mpath) as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            return None


    def manifest_compatible(
        manifest: dict,
        *,
        frames_dir: str,
        weights: str,
        sport: str,
        frames_in: int,
        frames_total: int,
        device: str = "cuda",
    ) -> bool:
        """A cache is reusable only if the run that produced it matches this run's inputs.

        ``device`` defaults to ``cuda`` so caches written before device-keying (no
``device``
        field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
cache
        (detections can differ numerically across devices), and vice-versa.

        ``frames_dir`` is the **already-canonical cache key** the caller stored in the
manifest
        (``run()`` passes its ``cache_key`` here, and writes the same value into the
manifest):
        ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
``video:<abspath>``
        key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
it here:
        ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
```

```
prepended,
        so a streaming cache would never match itself and every re-run would recompute
(issue #348).
        """
        if not manifest or manifest.get("version") != CACHE_VERSION:
            return False
        return (
            manifest.get("frames_dir") == frames_dir
            and manifest.get("weights") == weights
            and manifest.get("sport") == sport
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
            and manifest.get("device", "cuda") == device
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid: list = []
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break  # truncated trailing line (crash mid-flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
```

```
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)


    # ---------------------------------------------------------------------- #
    # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
    # ---------------------------------------------------------------------- #
    _STREAM_FRAME_FMT = "{:08d}.png"


    def synthetic_frame_path(index: int) -> str:
        """Stable, index-encoding frame name used by the streaming path.

        It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
        downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
        frames came from disk or from the in-memory stream. The streaming image loader
        inverts this name back to an index to fetch the decoded frame.
        """
        return _STREAM_FRAME_FMT.format(int(index))


    def build_windows(frames: list, frames_in: int):
        """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
        + checkpoint). Pure list math, identical for the disk and streaming paths.

        Returns a list of windows, each a list of ``frames_in`` consecutive paths
        (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
        caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
        ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
        """
        return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]


    class SequentialFrameStore:
        """In-memory sliding buffer over a forward-only frame reader, sized for the
        detector's sliding-window access pattern (peak O(window) frames, not O(video)).

        The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
        ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
        in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
        start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
        boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
        far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
        earliest index any not-yet-finished window can still ask for) and decode each source
        frame exactly once via the forward-only ``reader(index)``.
        """

        def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
            self._reader = reader
            self._total = int(total)
            self._window = max(1, int(window))
            self._buf: dict = {}
            # On a resumed run the detector's first needed frame is `start_index`; begin
            # pulling there so the reader can skip (seek past) the already-cached prefix
            # instead of decoding 0..start_index-1 just to throw them away.
            self._next = int(start_index)  # next index not yet pulled from the reader
            self._max_seen = int(start_index) - 1
            self._floor = int(start_index)  # lowest index we still keep (never evict below)
```

```
294
295         def get(self, index: int):
296             index = int(index)
297             if index < self._floor:
298                 raise KeyError(
299                     f"frame {index} already evicted (below floor {self._floor}); the access
"
300                     f"pattern must stay within a window of the highest frame seen"
301                 )
302             if index >= self._total:
303                 raise KeyError(f"frame {index} out of range (total {self._total})")
304             # Pull forward from the reader until the requested index is buffered.
305             while self._next <= index:
306                 self._buf[self._next] = self._reader(self._next)
307                 self._next += 1
308             if index > self._max_seen:
309                 self._max_seen = index
310             # Evict frames older than the earliest a still-running window could re-request:
311             # once we've seen index `m`, no future window starts before `m - (window-1)`.
312             new_floor = max(self._floor, self._max_seen - (self._window - 1))
313             for k in range(self._floor, new_floor):
314                 self._buf.pop(k, None)
315             self._floor = new_floor
316             return self._buf[index]
317
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326
327
328     def _bgr_to_pil_rgb(frame_bgr):
329         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
333         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
334         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
335         frame indistinguishable from the on-disk one to everything downstream.
336         """
337         import cv2
338         from PIL import Image
339
340         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
341
342
343     def _make_streamed_read_image(store, real_read_image):
344         """Build the ``read_image`` replacement used during a streaming detector pass.
345
346         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
347         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
348         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
349         without the WSL-only ``dataloaders`` package.
350         """
```

```
351
352         def _read_image(path, *args, **kwargs):
353             idx = _index_from_synthetic_path(path)
354             if idx < 0:
355                 return real_read_image(path, *args, **kwargs)
356             return _bgr_to_pil_rgb(store.get(idx))
357
358         return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
366         frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
370         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
371         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
372         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
373         ``osp.exists`` guard is sidestepped for synthetic stream paths.
374
375         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
376         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
377         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
378         reference).
379
380         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
381         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
382         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
383         original reader is restored on exit.
384         """
385         from dataloaders import dataset_loader as _dl
386
387         store = SequentialFrameStore(
388             frame_loader, total, window=window, start_index=start_index
389         )
390         real_read_image = _dl.read_image
391         # Rebind read_image in the consuming module for the duration of the detector pass;
392         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
393         _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type:
ignore[assignment]
394         try:
395             yield store
396         finally:
397             _dl.read_image = real_read_image  # type: ignore[assignment]
398
399
400     # ------------------------------------------------------------------------ #
401     # Inference
402     # ------------------------------------------------------------------------ #
403     class _PrefetchIter:
```

```
        """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).

        In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
        the cv2.imread shim live in this process and cannot be pickled to workers), so the
        CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
        measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
        A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
        release the GIL for the heavy parts, the store's forward-only access pattern is
        preserved (one consumer thread iterating sequentially), and the bounded queue caps
        memory at ``depth`` batches. A producer exception is re-raised in the consumer."""

        _END = object()

        def __init__(self, src, depth: int = 2):
            import queue
            import threading

            self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
            self._exc: "BaseException | None" = None

            def _fill() -> None:
                try:
                    for item in src:
                        self._q.put(item)
                except BaseException as e:  # noqa: BLE001 - surfaced in __next__
                    self._exc = e
                finally:
                    self._q.put(self._END)

            self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
            self._t.start()

        def __iter__(self):
            return self

        def __next__(self):
            item = self._q.get()
            if item is self._END:
                if self._exc is not None:
                    raise self._exc
                raise StopIteration
            return item


    def run(
        frames,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        num_workers: int = 4,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
        fresh: bool = False,
        frame_loader=None,
        source_key: "str | None" = None,
        device: str = "cuda",
        prefetch_batches: int = 0,
    ) -> str:
        """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
```

```
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
471         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
472           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
473           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
474           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
475           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
476           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
477           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
478           (the in-process shim + buffer can't cross worker processes).
479
480         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
481         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
482         after reboot still matches.
483         """
484         import time
485
486         import torch
487         from dataloaders import build_img_transforms, build_seq_transforms
488         from dataloaders.dataset_loader import ImageDataset
489         from torch.utils.data import DataLoader
490
491         # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
492         # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
493         # the mypy false positive where the main env resolves `trackers` to the Roboflow
package
494         # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
495         # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
496         from trackers import build_tracker  # type: ignore[attr-defined]
497         from utils import Center
498
499         from detectors import build_detector
500
501         streaming = frame_loader is not None
502
503         cfg = _build_cfg(weights, sport, device)
504         frames_in = int(cfg["model"]["frames_in"])
505         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508         if streaming:
509             # `frames` is already the ordered list of (synthetic) frame paths.
510             if not isinstance(frames, (list, tuple)):
511                 raise SystemExit(
512                     "streaming mode requires `frames` to be a list of frame paths"
513                 )
514             frames = list(frames)
515             frames_dir = source_key or "stream"
516         else:
517             frames_dir = frames
518             frames = sorted(
519                 glob.glob(osp.join(frames_dir, "*.png"))
520                 + glob.glob(osp.join(frames_dir, "*.jpg"))
521             )
```

```
        if limit:
            frames = frames[:limit]
        if len(frames) < frames_in:
            where = frames_dir if not streaming else "video stream"
            raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")

        # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
        samples = []
        for window in build_windows(frames, frames_in):
            annos = [
                {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
                for p in window
            ]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {},
            frames_dir=cache_key,
            weights=weights,
            sport=sport,
            frames_in=frames_in,
            frames_total=len(frames),
            device=device,
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(
                f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
            )
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": cache_key,
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "device": device,
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ----------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
```

```
                seq_transform_test = None
            ds = ImageDataset(
                cfg,
                remaining,
                input_wh,
                output_wh,
                transform=transform_test,
                seq_transform=seq_transform_test,
                is_train=False,
            )
            # Streaming forces single-process loading: the in-memory frame store + the
            # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
            loader_workers = 0 if streaming else num_workers
            loader = DataLoader(
                ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
            )
            detector = build_detector(cfg)

            from contextlib import ExitStack

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(
                f"detecting on {len(remaining)} remaining windows "
                f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
                f"source={'video-stream' if streaming else 'frames-dir'})..."
            )
            det_f = open(det_jsonl, "a")
            try:
                with ExitStack() as stack:
                    stack.enter_context(torch.no_grad())
                    # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
                    # in-memory decoded frame for that synthetic path; every other line of
                    # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
                    # The detector's first needed frame is `windows_done` (window i starts
at
                    # frame i), so prime the store there for a resume.
                    if streaming:
                        stack.enter_context(
                            _streaming_read_image_patch(
                                frame_loader,
                                len(frames),
                                window=frames_in,
                                start_index=windows_done,
                            )
                        )
                    # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
                    # a multi-worker DataLoader already overlaps via its worker processes).
                    batches = (
                        _PrefetchIter(loader, depth=prefetch_batches)
                        if prefetch_batches and loader_workers == 0
                        else loader
                    )
                    for bi, batch in enumerate(batches):
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [
                            list(t) for t in batch[-1]
```

```
                        ]  # [frame_in][batch] -> path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(
                                f"{done}/{windows_total} "
                                f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
                                f"elapsed {el:.0f}s | ETA {eta:.0f}s"
                            )
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2

```

```
        os.makedirs(frames_dir, exist_ok=True)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def _probe_frame_count(video_path: str) -> int:
        """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        cap.release()
        return n


    def make_video_frame_loader(video_path: str):
        """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

        ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
        long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
        detector's sliding-window order); it reads sequentially and only uses a frame-seek
to
        skip forward when a *resume* starts past the current position. Each ``cap.read()``
        returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
        yield (PNG is lossless), so the detector sees identical pixels to the disk path.

        Returns the count hint from the container so the caller can build the frame list;
        ``release`` closes the capture.
        """
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        state = {"pos": 0}  # next index cap.read() will return

        def frame_loader(index: int):
            index = int(index)
            if index < state["pos"]:
                raise RuntimeError(
                    f"streaming decode is forward-only; got index {index} after
{state['pos']}"
                )
```

```
            if index > state["pos"]:
                # Forward skip only happens when resuming past cached windows. Seek once.
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                state["pos"] = index
            ok, frame = cap.read()
            if not ok:
                raise RuntimeError(f"failed to decode frame {index} from {video_path}")
            state["pos"] += 1
            return frame

        def release():
            cap.release()

        return frame_loader, count_hint, release


    def run_video_streaming(
        video_path: str,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
        fresh: bool = False,
        device: str = "cuda",
        prefetch_batches: int = 0,
    ) -> str:
        """Fast path: decode ``video_path`` on the fly and feed frames straight to the
        detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
        dominates a long clip). Output is bit-identical to the frames-on-disk path; see
        ``run``'s docstring for why.
        """
        frame_loader, count_hint, release = make_video_frame_loader(video_path)
        try:
            if count_hint <= 0:
                raise SystemExit(
                    f"could not determine frame count for {video_path}; cannot stream "
                    f"(fall back to frame extraction with --frames_dir)"
                )
            frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
            source_key = "video:" + osp.abspath(video_path)
            return run(
                frame_list,
                weights,
                out_csv,
                sport,
                limit,
                batch_size=batch_size,
                num_workers=0,
                log_every_batches=log_every_batches,
                cache_dir=cache_dir,
                flush_every=flush_every,
                fresh=fresh,
                frame_loader=frame_loader,
                source_key=source_key,
                device=device,
                prefetch_batches=prefetch_batches,
            )
        finally:
            release()
```

```
    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument(
            "--video", help="video file; frames are extracted internally (in WSL)"
        )
        ap.add_argument(
            "--frames_out_dir", help="where to extract frames when --video is used"
        )
        ap.add_argument(
            "--stream-video",
            action="store_true",
            help="FAST PATH: decode --video on the fly and feed frames straight to "
            "the detector (no PNG extraction). Output-equivalent to the disk path.",
        )
        ap.add_argument("--weights", required=True)
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument(
            "--device",
            default="cuda",
            choices=["cuda", "mps", "cpu"],
            help="torch device for the detector (default cuda = the WSL parity path)",
        )
        ap.add_argument(
            "--limit", type=int, default=0, help="cap #frames (for quick tests)"
        )
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument(
            "--prefetch-batches",
            type=int,
            default=0,
            help="streaming mode: overlap CPU batch assembly with GPU inference via a "
            "bounded producer thread of this queue depth (0 = off, the parity default)",
        )
        ap.add_argument(
            "--log-every-batches", type=int, default=50, help="progress log cadence"
        )
        ap.add_argument(
            "--cache-dir",
            default=None,
            help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
        )
        ap.add_argument(
            "--flush-every",
            type=int,
            default=1000,
            help="fsync the detector cache every N windows (bounds reboot loss)",
        )
        ap.add_argument(
            "--fresh", action="store_true", help="ignore any existing cache and start over"
        )
        args = ap.parse_args()

        if args.stream_video:
            if not args.video:
                raise SystemExit("--stream-video requires --video")
            run_video_streaming(
                args.video,
                args.weights,
                args.out,
                args.sport,
                args.limit,
                batch_size=args.batch_size,
```

```
                log_every_batches=args.log_every_batches,
                cache_dir=args.cache_dir,
                flush_every=args.flush_every,
                fresh=args.fresh,
                device=args.device,
                prefetch_batches=args.prefetch_batches,
            )
            return

        frames_dir = args.frames_dir
        if args.video:
            frames_dir = extract_frames(
                args.video, args.frames_out_dir or (args.video + "_frames")
            )
        if not frames_dir:
            raise SystemExit("provide --frames_dir or --video")
        run(
            frames_dir,
            args.weights,
            args.out,
            args.sport,
            args.limit,
            batch_size=args.batch_size,
            num_workers=args.num_workers,
            log_every_batches=args.log_every_batches,
            cache_dir=args.cache_dir,
            flush_every=args.flush_every,
            fresh=args.fresh,
            device=args.device,
        )


    if __name__ == "__main__":
        main()

```

4. user (2026-07-03T19:52:20.081Z)

```
      """Linux-native WASB shuttle-trajectory runner (no WSL).

      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
      Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
      see the acceptance gate below) ? but WITHOUT the WSL transport:

        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
        * weights / repo / python / scratch come from the **environment** first
          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.

      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
```

```
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34      not the numbers.
35      """
36
37      import logging
38      import os
39      import shutil
40      import subprocess
41      from dataclasses import dataclass
42      from typing import List, Optional, Tuple
43
44      from backend.config.models import IndexingConfig
45      from backend.pipeline.detectors.base import DetectorRunner
46      from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51      logger = logging.getLogger(__name__)
52
53      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57      # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58      # so the "cuda True/False" string they parse can never drift apart.
59      _CUDA_PROBE_CODE = (
60          "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61      )
62
63
64      @dataclass
65      class NativeWasbConfig:
66          """Resolved settings for a Linux-native WASB run.
67
68          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70          defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71          """
72
73          repo_dir: str = "~/models/WASB-SBDT"  # native clone of nttcom/WASB-SBDT
74          python_bin: str = (
```

```
75              "python"  # the `wasb` conda env python (invoked by path, no activate)
76          )
77          weights_path: str = ""  # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78          sport: str = "badminton"
79          device: str = "cuda"  # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80          scratch_dir: str = ""  # frame-extraction scratch (env); "" = next to the output dir
81          timeout_sec: int = 1800  # 30 min; a hung GPU call is killed (0 = no timeout)
82          keep_frames: bool = (
83              False  # retain the decoded frame cache after success (debug only)
84          )
85          # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86          # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87          # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88          stream_video: bool = True
89          # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90          # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91          batch_size: int = 0
92          prefetch_batches: int = 0
93
94          @classmethod
95          def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96              if isinstance(idx_cfg, dict):
97                  idx_cfg = IndexingConfig(**idx_cfg)
98              w = idx_cfg.wasb
99              env = os.environ.get
100             # stream_video is tri-state in config: None/absent => native default (stream);
an
101             # explicit True/False still wins (e.g. False for a container cv2 can't seek).
102             sv = w.stream_video
103             return cls(
104                 repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
105                 python_bin=env("WASB_PYTHON") or w.python_bin,
106                 weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
107                 sport=w.sport,
108                 device=env("WASB_DEVICE") or w.device,
109                 scratch_dir=env("WASB_SCRATCH")
110                 or env("RALLY_SCRATCH")
111                 or env("TMPDIR")
112                 or "",
113                 timeout_sec=w.timeout_sec,
114                 keep_frames=w.keep_frames,
115                 stream_video=(True if sv is None else bool(sv)),
116                 batch_size=int(w.batch_size or 0),
117                 prefetch_batches=int(w.prefetch_batches or 0),
118             )
119
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
123
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
127             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
128             self._cuda_available: Optional[bool] = None
129
130         # --- low-level native exec (the single place tests patch)
-------------------------
```

```
131         def _run(
132             self, argv: List[str], cwd: Optional[str] = None
133         ) -> subprocess.CompletedProcess:
134             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
135             return subprocess.run(
136                 argv,
137                 cwd=cwd,
138                 capture_output=True,
139                 text=True,
140                 timeout=(self.cfg.timeout_sec or None),
141             )
142 
143         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
144         def _probe_cuda(self) -> bool:
145             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
146             (missing python / timeout / import error) is treated as 'no CUDA'."""
147             try:
148                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
149             except (FileNotFoundError, subprocess.TimeoutExpired):
150                 return False
151             return res.returncode == 0 and "cuda True" in (res.stdout or "")
152 
153         def _cuda_is_available(self) -> bool:
154             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
155             if self._cuda_available is None:
156                 self._cuda_available = self._probe_cuda()
157             return self._cuda_available
158 
159         def _effective_device(self) -> str:
160             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162             if self.cfg.device == AUTO:
163                 return DeviceContext(AUTO, self._cuda_is_available()).effective
164             return self.cfg.device
165 
166         def _repo_src(self) -> str:
167             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168 
169         def _copy_infer_script(self) -> bool:
170             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171             repo_src = self._repo_src()
172             try:
173                 os.makedirs(repo_src, exist_ok=True)
174                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175                 return True
176             except OSError as e:
177                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178                 return False
179 
180         def _rm_rf(self, path: Optional[str]) -> None:
181             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182             if path:
183                 shutil.rmtree(path, ignore_errors=True)
184 
185         def healthcheck(self) -> Tuple[bool, str]:
186             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187             if not self.cfg.weights_path:
```

```
188                 return False, (
189                     "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
190                 )
191             weights = os.path.expanduser(self.cfg.weights_path)
192             if not os.path.exists(weights):
193                 return False, f"WASB weights not found at {weights}"
194             repo_src = self._repo_src()
195             if not os.path.isdir(repo_src):
196                 return False, (
197                     f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198                     f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
199                 )
200             # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201             # argparse rc=2 from wasb_infer.py at run time.
202             if self.cfg.device not in VALID_DEVICES:
203                 return False, (
204                     f"unknown device {self.cfg.device!r} ? valid: {',
'.join(VALID_DEVICES)}."
205                 )
206             try:
207                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
208             except FileNotFoundError:
209                 return (
210                     False,
211                     f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
212                 )
213             except subprocess.TimeoutExpired:
214                 return False, "native torch healthcheck timed out."
215             out = (res.stdout or "").strip()
216             if res.returncode != 0:
217                 return (
218                     False,
219                     f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
220                 )
221             # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
222             self._cuda_available = "cuda True" in out
223             ctx = DeviceContext(self.cfg.device, self._cuda_available)
224             if ctx.cuda_required_but_missing:
225                 # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
226                 # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
227                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
228             if ctx.fell_back:
229                 logger.warning(
230                     "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
231                     "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
232                     out,
233                 )
234             return True, out
235
236         def run_predict(
237             self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
238         ) -> Optional[str]:
239             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
240
241             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
242             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
```

```
            same downstream contract, so the parity comparison is apples-to-apples.
            """
            output_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_dir, exist_ok=True)
            # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
            norm = str(video_win_path).replace("\\", "/")
            stem, _ext = os.path.splitext(os.path.basename(norm))
            # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
            key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
            out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

            if not self._copy_infer_script():
                logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
                return None

            weights = os.path.expanduser(self.cfg.weights_path)
            argv = [
                self.cfg.python_bin,
                "wasb_infer.py",
                "--video",
                os.path.abspath(str(video_win_path)),
            ]
            frames_dir: Optional[str] = None
            if self.cfg.stream_video:
                # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
                argv.append("--stream-video")
            else:
                scratch = self.cfg.scratch_dir or output_dir
                frames_dir = os.path.join(scratch, f"{key}_frames")
                argv += ["--frames_out_dir", frames_dir]
            # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
            # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
            device = self._effective_device()
            argv += [
                "--weights",
                weights,
                "--sport",
                self.cfg.sport,
                "--device",
                device,
                "--out",
                out_csv,
            ]
            # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
            # byte-parity guarantee) is unchanged when the knobs are unset.
            if self.cfg.batch_size:
                argv += ["--batch-size", str(self.cfg.batch_size)]
            if self.cfg.prefetch_batches:
                argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]

            logger.info(
                "Running native WASB inference (device=%s%s) on %s ...",
                device,
                " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
                os.path.basename(norm),
            )
            try:
                res = self._run(argv, cwd=self._repo_src())
            except FileNotFoundError:
                logger.error(
```

```
303                     "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304                 )
305                 return None
306             except subprocess.TimeoutExpired:
307                 logger.error(
308                     "WASB native inference timed out after %ss.", self.cfg.timeout_sec
309                 )
310                 return None
311             if res.returncode != 0:
312                 logger.error(
313                     "WASB native inference failed:\n%s",
314                     (res.stderr or res.stdout or "")[-2000:],
315                 )
316                 return None
317             if not os.path.exists(out_csv):
318                 logger.error(
319                     "WASB native finished but expected CSV not found at %s", out_csv
320                 )
321                 return None
322             logger.info("WASB trajectory CSV: %s", out_csv)
323 
324             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
327             if not self.cfg.keep_frames and frames_dir:
328                 logger.info(
329                     "Cache hygiene: removing frame cache for %s "
330                     "(set indexing.wasb.keep_frames=true to retain).",
331                     key,
332                 )
333                 self._rm_rf(frames_dir)
334             return out_csv
335 
336         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338         trajectory_to_action_windows = staticmethod(
339             TrackNetRunner.trajectory_to_action_windows
340         )
341 
```

5. assistant (2026-07-03T19:52:22.538Z)

Now let me check the config models, presets, and Dockerfile to ground the plan.

6. user (2026-07-03T19:52:24.916Z)

```
1       """Pydantic models for the application configuration.
2 
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5 
6       Unknown / typo'd config keys are rejected at load time via extra="forbid".
7       """
8 
9       from __future__ import annotations
10 
11      from typing import Any, List, Literal, Optional
12 
13      from pydantic import BaseModel, Field, field_validator, model_validator
14 
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
```

```
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          pts_max_gap_seconds: float = Field(
31              default=10.0,
32              ge=0.0,
33              description=(
34                  "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                  "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                  "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                  "recordings, and other encoders may use different GOP sizes ? tune "
38                  "this threshold for your recording device."
39              ),
40          )
41          relevance: ValidationRelevanceConfig = Field(
42              default_factory=ValidationRelevanceConfig
43          )
44
45
46      class TrackNetConfig(BaseModel):
47          wsl_distro: str = "Ubuntu"
48          repo_dir: str = "~/models/TrackNetV4"
49          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50          conda_env: str = "TrackNetV4"
51          weights_path: str = ""
52          queue_length: int = 5
53          stage_in_wsl: bool = True
54          wsl_stage_dir: str = "~/clips"
55          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56          velocity_threshold: float = 0.02
57          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58          # decoded frame cache) are deleted automatically ? they are pure intermediates,
59          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61          keep_frames: bool = False
62          # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63          # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64          # 0 = no extra floor (the headroom guard still applies).
65          min_free_gb: float = 0.0
66
67
68      class WasbIndexConfig(BaseModel):
69          """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72          these knobs actually take effect ? previously this whole block was silently dropped
73          because nested config sections defaulted to extra='ignore'.
74          """
75
76          wsl_distro: str = "Ubuntu"
77          repo_dir: str = "~/models/WASB-SBDT"
78          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
```

```
79          conda_env: str = "wasb"
80          weights_path: str = (
81              "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82          )
83          sport: str = "badminton"
84          wsl_stage_dir: str = "~/clips"
85          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86          velocity_threshold: float = 0.02
87          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
88          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89          # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90          # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91          device: str = "cuda"  # auto | cuda | mps | cpu
92          python_bin: str = (
93              "python"  # the `wasb` conda env python (invoked by path, no `conda activate`)
94          )
95          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98          keep_frames: bool = False
99          # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100         # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101         # 0 = no extra floor (the headroom guard still applies).
102         min_free_gb: float = 0.0
103         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108         # Tri-state:
109         #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110         #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111         #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112         stream_video: Optional[bool] = None
113         # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114         # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115         # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116         # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117         # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118         batch_size: Optional[int] = None
119         # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120         # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121         prefetch_batches: Optional[int] = None
122
123
124     class FusionConfig(BaseModel):
125         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
```

```
126
127         Every default reproduces control behaviour: the fusion segmenter persists the
128         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
129         true ? the person-cue features, but it does NOT gate the served handoff unless a
130         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
131         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
132         supplied ? off-court persons (spectators / adjacent courts) are excluded.
133         """
134
135         # Which trajectory substrate seeds the windows (shuttle stays primary).
136         substrate: Literal["wasb", "tracknet"] = "wasb"
137         # Windowing velocity threshold for the fusion family (the engine reads
138         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
139         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
140         velocity_threshold: float = 0.02
141         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
142         # enabled ? so the default path never imports torch / touches the GPU.
143         cue_flags: dict[str, bool] = Field(default_factory=dict)
144         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
145         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
146         min_crossings: int = Field(default=0, ge=0)
147         # Served Gate B: when the person cue is on, drop windows with median in-court
players
148         # < this. 0 = OFF.
149         min_players: int = Field(default=0, ge=0)
150         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
151         court_polygon: Optional[list[list[float]]] = None
152         # Net line for the person two-sided-motion split (person features only ? the shuttle
153         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
154         net_axis: Literal["x", "y"] = "y"
155         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
156
157
158     class IndexingConfig(BaseModel):
159         default_segmenter: str = "yolo_hybrid"
160         use_gpu: bool = True
161         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
162         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
163         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
164         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
165         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
166         detector_impl: str = "auto"
167         motion_threshold: float = 0.08
168         min_segment_duration: float = 3.0
169         dead_time_merge_gap: float = 2.0
170         chunk_duration: float = 90.0
171         chunk_overlap: float = 10.0
172         detector_model: str = "fasterrcnn_resnet50_fpn"
173         detector_score_threshold: float = 0.5
174         yolo_velocity_threshold: float = 0.15
175         yolo_frame_skip: int = 3
176         yolo_tracker: str = "bytetrack.yaml"
177         yolo_prefetch_workers: int = 2
178         min_action_window_duration: float = 2.0
179         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
180         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
181         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
```

```
swallow
182         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
183         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
184         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
185         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
186         max_window_duration: float = 6.0
187         window_min_split_gap: float = (
188             0.5  # min internal gap (s) that qualifies as a split point
189         )
190         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
191         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
192         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
193         # Only cuts (recall can't drop). OFF by default until measured/promoted.
194         window_hard_cap: bool = False
195         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
196         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
197         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
198         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
199         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
200         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
201         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
202         window_inactivity_end: float = 1.5
203         inactivity_rally_thresh: float = 0.20
204         inactivity_min_density: float = 0.30
205         # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
206         # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
207         # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
208         # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
209         # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
210         # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
211         # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
212         # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
213         # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
214         inactivity_temporal_density: bool = True
215         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
216         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
217         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
218         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
219         # (logged + flagged on the window) so the degenerate single-window outcome is
```

```
avoided and the
220         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
221         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
222         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
223         window_blob_fallback: bool = False
224         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
225         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
226         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
227         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
228         window_blob_factor: float = 4.0
229         log_yolo_candidates: bool = True
230         store_yolo_candidates: bool = True
231         ai_handoff_padding: float = 2.0
232         ai_max_workers: int = 5
233         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
234         # chunks of a high-bitrate (4K) source blow past the provider upload cap
235         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
236         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
237         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
238         # Matches gemini_refine's --clip-scale-width default.
239         ai_chunk_scale_width: int = 1280
240         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
241         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
242         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
243         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
244         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
245         skip_ai_handoff: bool = False
246         # Optional debug knob: trim every video to the first N seconds before processing.
247         debug_duration: Optional[float] = None
248
249         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
250         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
251         fusion: FusionConfig = Field(default_factory=FusionConfig)
252
253         @field_validator("default_segmenter")
254         @classmethod
255         def segmenter_must_be_registered(cls, v: str) -> str:
256             # Lazy import: the segmenter registry imports modules that may import
257             # config, so we resolve it at validation time rather than at module load.
258             from backend.pipeline.segmenters.base import segmenter_registry
259
260             if segmenter_registry.get(v) is None:
261                 available = sorted(segmenter_registry.list_available().keys())
262                 raise ValueError(
263                     f"Segmenter '{v}' is not registered. Available: {', '.join(available)}"
264                 )
265             return v
266
267         @model_validator(mode="after")
268         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
269             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
270             # step makes `while t < duration: t += step` loop forever, growing memory before
any
271             # GPU work. Reject the bad config at load time instead.
```

```
272             if self.chunk_overlap >= self.chunk_duration:
273                 raise ValueError(
274                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
275                     f"({self.chunk_duration}); otherwise chunking never advances."
276                 )
277             return self
278 
279 
280     class OutputConfig(BaseModel):
281         default_highlights_name: str = "highlights.mp4"
282         db_name: str = "sports_indexer.db"
283         # Directory where the DB, highlight reels, and processed clips are written.
284         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
285         output_dir: str = "output"
286 
287 
288     class WatermarkConfig(BaseModel):
289         """Branding overlay burned into each highlight clip during the per-clip re-encode
290         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
291         (under `stitching.watermark` in config.json) to turn it off. The text is fully
292         configurable. The concat stage stays stream-copy (-c copy)."""
293 
294         enabled: bool = True
295         text: str = "Generated by Khelsutra.guru"
296         logo_path: str = ""  # optional transparent PNG overlay; "" = no logo
297         font_path: str = ""  # optional .ttf; "" = use the fontconfig `font` family below
298         font: str = "Sans"  # fontconfig family used when font_path is empty
299         font_size_ratio: float = Field(
300             default=0.035, gt=0.0, le=0.5
301         )  # text height as a fraction of frame height
302         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
303         box: bool = True  # faint backing box behind the text for legibility
304         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
305         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
306             "bottom-right"
307         )
308 
309 
310     class StitchingConfig(BaseModel):
311         begin_padding: float = 0.5
312         end_padding: float = 0.5
313         use_cache: bool = False
314         min_shot_count: int = 1
315         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
316         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
317         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
318         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
319         # local detector over-fires and its false positives are mostly SHORT (motion blips,
320         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
321         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
322         min_rally_duration: float = Field(default=0.0, ge=0.0)
323         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
324 
325 
326     class LoggingConfig(BaseModel):
327         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
328 
329         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
330         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
```

```
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]

        # --- M9 edge auth (Cloudflare Access, D9) ---
        # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
        # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
        # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
        # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
        # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
        # application's audience tag. NO secrets here ? these are public identifiers.
        edge_auth_enabled: bool = False
        cf_team_domain: str = (
            ""  # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
        )
        cf_access_aud: str = ""  # the CF Access application AUD tag the token must carry
        # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
        # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
        # OFF ? Cf-Access is a header, not a cookie.)


    class StorageConfig(BaseModel):
        """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
        behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
        + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
        Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
        **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
        (boto3/google auth chains supply credentials). See ``backend/storage/``."""

        backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
        # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
        output_root: str = ""
        # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
        # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
```

```
382         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
383         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
384         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
385         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
386         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
387         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
388         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
389         input_root: str = ""
390         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
391         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
392         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
393         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
394         # the legacy flat layout is used unchanged.
395         single_folder_per_video: bool = False
396         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
397         workspace_root: str = ""
398         # Cloud namespace prefix under the root so per-video folders sit tidily beside
399         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
400         workspace_prefix: str = "workspaces"
401         local: dict[str, Any] = Field(default_factory=dict)
402         s3: dict[str, Any] = Field(default_factory=dict)
403         gcs: dict[str, Any] = Field(default_factory=dict)
404         gdrive: dict[str, Any] = Field(default_factory=dict)
405         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
406         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
407         gdrive_api: dict[str, Any] = Field(default_factory=dict)
408
409
410     class DeploymentConfig(BaseModel):
411         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
412         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
413
414         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
415         behaves exactly as before this section existed. A profile is opt-in (``config.json``
416         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
417         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
418         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
419         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
420
421         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
422
423         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
424         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
425         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
426         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
427         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
428
```

```
429
430     class BillingConfig(BaseModel):
431         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
432
433         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
434         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
435         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
436         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
437         inserts a real, calibrated version and points ``pricebook_version`` at it.
438
439         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
440         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
441         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
442
443         enabled: bool = False
444         pricebook_version: str = (
445             "placeholder-v0"  # the seeded $0 version; owner flips to a real one
446         )
447         fx_inr_per_usd: float = 83.0  # USD?INR display rate (Bangalore market)
448         margin: float = 0.0  # resale markup (?0); 0 = bill at cost
449
450
451     class FlywheelConfig(BaseModel):
452         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
453         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
454         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
455         the owned model climbs toward the D11 ship floor.
456
457         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
458         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
459         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
460         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
461         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
462         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
463         owner/counsel/GPU scope. Pairs with M9-consent."""
464
465         enabled: bool = False
466         workspace_prefix: str = (
467             "flywheel"  # per-video workspace prefix for captured artifacts
468         )
469         shadow_eval: bool = True  # run the owned-vs-Gemini shadow comparison on capture
470         min_promote_n: int = 5  # promote_if_served_safe `min_n` (the served-safe gate)
471
472
473     class AppConfig(BaseModel):
474         """Root configuration object.
475
476         All runtime configuration should be accessed via an instance of this class
477         (or the loader) rather than raw dict lookups.
```

```
478         """
479
480         sport: Sport = "badminton"
481         ai_provider: str = "gemini"
482         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
483         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
484         ai_model: str = "gemini-flash-latest"
485
486         validation: ValidationConfig = Field(default_factory=ValidationConfig)
487         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
488         output: OutputConfig = Field(default_factory=OutputConfig)
489         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
490         security: SecurityConfig = Field(default_factory=SecurityConfig)
491         logging: LoggingConfig = Field(default_factory=LoggingConfig)
492         storage: StorageConfig = Field(default_factory=StorageConfig)
493         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
494         billing: BillingConfig = Field(default_factory=BillingConfig)
495         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
496
497         # Reject unknown keys ? typed config migration is complete (D3).
498         model_config = {
499             "extra": "forbid",
500             "validate_assignment": True,
501         }
502
503
504     # Convenience type alias for callers that want to annotate
505     Config = AppConfig
506
```

7. user (2026-07-03T19:52:25.360Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12      #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13      #
14      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15      #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16      #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20      ENV DEBIAN_FRONTEND=noninteractive \
21          PYTHONUNBUFFERED=1 \
```

```
        PIP_NO_CACHE_DIR=1 \
        CONDA_DIR=/opt/conda \
        WASB_REPO_DIR=/opt/models/WASB-SBDT

    # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
    RUN apt-get update && apt-get install -y --no-install-recommends \
            ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
            python3 python3-pip python3-venv \
        && rm -rf /var/lib/apt/lists/*

    # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
    # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
    # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
    # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
    # MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
    # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
    ENV MAMBA_ROOT_PREFIX=/opt/conda
    RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
            | tar -xj -C /usr/local bin/micromamba \
        && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
        && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
        && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
            torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
        && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
        && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
            hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"

    # --- Main application env ---
    WORKDIR /app
    COPY pyproject.toml README.md ./
    COPY requirements-trackers.txt ./
    COPY backend ./backend
    COPY training ./training
    # Install with the [storage-gcs] extra: the cloud worker PULLS the gs:// input and
PUSHES the
    # gs:// outputs/done-marker through backend.storage (google-cloud-storage is lazy-
imported there).
    # Without it the job dies at the first gs:// fetch with `ModuleNotFoundError:
google.cloud` ? the
    # real-Cloud-Run failure the mock-client tests can't see (the bucket I/O is faked in
tests).
    # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
    # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
    # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
    # single-sourced in requirements-trackers.txt.
    RUN python3 -m pip install --upgrade pip \
        && python3 -m pip install -r requirements-trackers.txt \
        && python3 -m pip install ".[storage-gcs]"

    # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
    # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
```

```
69      # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
70      # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
71      # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
72      ENV PYTHONUNBUFFERED=1 \
73          WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
74          WASB_REPO_DIR=/opt/models/WASB-SBDT \
75          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
76          RALLY_DETECTOR_IMPL=native
77
78      # The dispatcher (CloudRunCompute) supplies the per-execution args:
79      #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
80      ENTRYPOINT ["python3", "-m", "backend.worker"]
81
```

8. assistant (2026-07-03T19:52:29.397Z)

I have everything I need. Let me check the presets file for the cloud-serving preset to confirm the current batch/prefetch opt-in values.

9. user (2026-07-03T19:52:31.547Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
```

```
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55              # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56              # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57              "indexing": {
58                  "detector_impl": "native",
59                  "skip_ai_handoff": False,
60                  "wasb": {"batch_size": 64, "prefetch_batches": 4},
61              },
62              "storage": {"backend": "gcs"},
63          },
64          "cpu-mock": {
65              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
67              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
68              "storage": {"backend": "local"},
69          },
70      }
71
72
73      def is_known_profile(profile: str) -> bool:
74          """True for a non-empty, recognised profile name."""
75          return bool(profile) and profile in PROFILE_PRESETS
76
77
78      def preset_for(profile: str) -> dict[str, Any]:
79          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
80          module-level table), or ``{}`` for ``""`` / an unknown profile."""
81          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82
83
84      def _truthy(val: Optional[str]) -> bool:
85          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
86          if val is None:
87              return False
88          return val.strip().lower() in {"1", "true", "yes", "on"}
89
90
```

```
91      def _cuda_available() -> bool:
92          """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
93          ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
94          try:
95              # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
96              # and optional). Only reached when a caller opts into probe_cuda.
97              import torch
98
99              return bool(torch.cuda.is_available())
100         except Exception:
101             return False
102
103
104     def probe_cuda_available() -> bool:
105         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
106
107         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
108         that *wants* the GPU heads-up can pass the result in without the startup module
importing
109         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
110         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
111         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
112         error that could false-fail."""
113         return _cuda_available()
114
115
116     def suggest_profile(
117         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
118     ) -> str:
119         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
120
121         Signals, in priority order:
122
123           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
124           - ``CI`` truthy                        ? ``"cpu-mock"``
125           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
126           - otherwise                            ? ``""`` (no strong suggestion)
127
128         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
129         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
130         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
131         """
132         e = os.environ if env is None else env
133         if e.get("K_SERVICE"):
134             return "cloud-serving"
135         if _truthy(e.get("CI")):
136             return "cpu-mock"
137         if probe_cuda and _cuda_available():
138             return "truly-local"
139         return ""
140
```

10. assistant (2026-07-03T19:54:12.954Z)

I now have full grounding. Let me write the definitive optimization plan.

WASB-on-L4 Inference Optimization Plan

> Scope: Phase-2 (WASB shuttle detection) on the Cloud Run L4 job. Baseline (first real L4 run, 2026-07-03): ~7.5-min 60fps clip ? **727.9s Phase-2 (~0.6× realtime), GPU util <1% the entire time.** Hardware: L4 (Ada sm_89, 23GB), 8 vCPU, 32GB. The `cloud-serving` preset already opts into `batch_size=64, prefetch_batches=4`.

0. The one-line diagnosis (read this first)

This is not a GPU-kernel problem. It is a CPU-feed-starvation problem. The evidence is unambiguous: GPU util sat **<1%** for 727.9s. A GPU that is 99%+ idle is not the bottleneck ? it is *waiting*. The pipeline is bound by the single-threaded CPU work that assembles each batch, and secondarily by the torch 1.11 startup/JIT-warmup story. Every lever below is ranked against that reality.

Two structural facts from the code that create the starvation:

1. **Streaming forces `num_workers=0`** (`wasb_infer.py:597`, `run_video_streaming` hardcodes `num_workers=0`). The in-memory `SequentialFrameStore` + the `read_image` monkeypatch live in-process and can't be pickled to DataLoader workers. So per-frame `cv2` decode ? `_bgr_to_pil_rgb` (cvtColor) ? PIL ? `ToTensor` all runs on **one** CPU core, inline with the GPU.
2. **Windows overlap by `frames_in - 1`**, so each source frame is decoded once (good ? the store handles that) but re-transformed into a tensor for **every window it appears in**. For `frames_in=3` that's ~3× the PIL?ToTensor work per source frame. This transform cost, single-threaded, is almost certainly the wall.

The prefetch thread (`_PrefetchIter`) is the right instinct but it only overlaps **one** producer thread with the GPU. If assembling a batch takes longer than the GPU forward (which the <1% util says it does, by ~100×), a single producer thread cannot fill the pipe ? you need *parallelism* in the CPU feed, not just *overlap*.

---

1. Ordered action list

Ranked by (expected_win × parity_safety × 1/effort). Grouped by shipping disposition.

A. Ship-as-default ? parity-safe, high-confidence wins

These do not change per-frame numerics (same forward, same transform, same input tensors ? only *when* and *on how many cores* the CPU work happens). They are parity-safe by construction.

#	Lever	Why it wins	Effort	Parity
A1	**Parallelize the CPU feed for the streaming path.** Today streaming is locked to `num_workers=0`. The fix is a **multi-threaded batch feeder** (a small thread pool over `Dataset.__getitem__`, not multiprocessing) so decode+PIL+ToTensor for several windows run concurrently on the 8 vCPUs. cv2/PIL/numpy release the GIL for the heavy work (the `_PrefetchIter` docstring already notes this), so threads scale here.	Med	**Safe** ? identical tensors, only produced on more cores. Order preserved by re-sequencing in the consumer.
A2	**Raise `prefetch_batches` and make the producer a pool, not a single thread.** `_PrefetchIter` currently runs *one* producer thread. Generalize it to N producer threads feeding the bounded queue (each pulling a disjoint stripe of window indices, results reordered). This is the concrete mechanism for A1.	Med	**Safe** ? same reason as A1.
A3	**Confirm & keep `batch_size=64` ? but do NOT expect it to be the win.** Bigger batches only help *once the feed is unblocked*. On a feed-starved pipeline, batch 64 mostly changes the flush granularity, not throughput. Keep it (it's free and parity-safe), but A1/A2 are what actually move the needle.	Done	**Safe** (already asserted).

A1/A2 are the real fix. Everything else is secondary.

B. Ship-as-opt-in ? accepts a new baseline (numerics may shift; NOT default)

These are likely large wins but they **change detector numerics** and therefore **break the D5/D16 byte-parity test**. They must be explicit opt-in knobs, gated behind a re-run of the parity acceptance gate that *re-baselines* the golden output.

#	Lever	Why it might win	Parity impact
B1	**FP16 / autocast inference**	Ada L4 has fast FP16/TF32 tensor cores; halves memory bandwidth. But the GPU is <1% busy ? this helps ~nothing until the feed is fixed, and even then the kernel isn't the bottleneck.	**BREAKS parity** ? different rounding ? different heatmap peaks ? different sub-pixel x,y. Opt-in only.
B2	**`torch.backends.cudnn.benchmark = True` + TF32**	Lets cuDNN pick faster algos; TF32 matmuls on Ada.	**BREAKS parity** ? algo selection + TF32 mantissa truncation both perturb output. Opt-in only.
B3	**NVDEC hardware decode** (GPU-side video decode via `torchaudio`/`PyNvVideoCodec`/`decord`)	Moves the decode off the 8 vCPUs entirely ? directly attacks the feed bottleneck. Potentially large.	**BREAKS parity** ? NVDEC's YUV?RGB colorspace conversion is **not** bit-identical to cv2's `cvtColor(BGR2RGB)` round-trip that the parity guarantee is pinned to. The code's whole `_bgr_to_pil_rgb` invariant (documented "max|diff|=0" vs the PNG round-trip) is with cv2. Opt-in only.

C. Measure-first ? promising but must be A/B'd before disposition

#	Lever	What to measure
C1	**The torch cu113 ? cu12x bump** (see §2). Big potential *startup/warmup* win + native sm_89 kernels.	Whether it's default-safe (it *should* be numerically identical run-to-run on the same GPU, but a torch minor bump can shift cuDNN defaults) ? see §2 & §5.
C2	**Batch-transform vectorization** ? do the PIL?ToTensor step on the whole batch at once, or on GPU (`torchvision.transforms.v2` / move ToTensor to CUDA). Attacks the per-frame CPU cost directly.	Whether the GPU-side transform is bit-identical to CPU ToTensor (float division). Likely NOT ? treat as opt-in if it changes bytes.
C3	**Skip redundant re-transforms across overlapping windows** ? cache the transformed tensor per source frame and assemble windows from the cache instead of re-running ToTensor `frames_in` times. Pure dedup of identical work.	Verify the assembled window tensor is byte-identical (it should be ? same input, same transform). If so, this is a **parity-safe default** and belongs in bucket A. Measure to confirm the ~`frames_in`× transform saving is real.

D. Reject

Lever	Why rejected
More GPUs / bigger GPU (A100/H100)	The L4 is <1% utilized. Throwing more GPU at a CPU-bound job is pure waste of spend (violates D8 cost discipline).
`torch.compile` / TensorRT export	Kernel-level optimization of a kernel that is already 99% idle. Zero wall-clock benefit until the feed is fixed; high effort; parity-breaking.
Increasing `batch_size` beyond 64 as the primary fix	Feed-bound; won't help (§3).

---

2. The torch cu113 ? cu12x bump (specific recommendation)

Recommended target: `torch==2.4.1+cu121` (with `torchvision==0.19.1+cu121`), Python 3.10, on the `nvidia/cuda:12.4.1-runtime` base already in the Dockerfile.

Rationale, concretely:
- The current WASB env is **torch 1.11.0+cu113 on py3.8** (Dockerfile line 44). **cu113 has no compiled kernels for Ada sm_89.** On an L4, a cu113 torch runs sm_89 through **PTX JIT compilation at first kernel launch** ? every unseen kernel shape is JIT-compiled by the driver on first use. This is a real, and likely large, contributor to the "GPU <1%" story: early in the run the GPU is stalled JIT-compiling rather than computing, and the process may be falling back

to slow paths. cu121+ ships **native sm_89 cubins** ? no JIT, correct fast kernels.
- The base image is **already `cuda:12.4.1`** (Dockerfile line 18), so the host CUDA userland supports cu121/cu124 wheels. The cu113 pin is a legacy artifact from the RTX-2070/WSL era, not a requirement of this image.
- 2.4.1 is a mature, widely-deployed release with py3.10 wheels; it's a safer landing than the very latest.

Is it the single biggest *safe* win? ? **Very likely yes, for the "GPU <1% + JIT" symptom, IF the parity gate passes.** The <1% util is over-determined (feed starvation *and* JIT), so the bump alone won't get you to realtime, but killing the sm_89 JIT is plausibly the biggest *single* discrete change and it *may* be numerically safe. **The catch:** a torch 1.11?2.4 jump is a large surface, and it can silently change cuDNN default algorithm selection and default math modes. **So it is "measure-first, then decide default vs opt-in" (bucket C1), not an assumed default.** Requires (a) rebuilding the WASB conda env against sm_89-capable wheels, and (b) passing the parity gate in §4. If parity passes ? ship as default (it's the highest-value safe change). If parity fails ? it must ride as an opt-in alongside B1/B2, because whatever shifted the bytes is a numeric change.

WASB weights are torch 1.11-era `.pth.tar`; confirm they load under 2.4 (they should ? plain `state_dict`; add a `weights_only=False` shim if the checkpoint pickles non-tensor objects).

---

3. The one lever most likely to be the true bottleneck

It is the CPU decode/transform feed, not the GPU kernel. State this plainly in the report:

> With GPU utilization measured at <1% for the full 727.9s, the WASB forward pass is **not** on the critical path. The pipeline is **CPU-feed-bound**: single-threaded (`num_workers=0`, forced by streaming) per-frame decode ? BGR?RGB ? PIL ? ToTensor cannot produce batches fast enough to keep the L4 busy, and overlapping windows re-run the transform ~`frames_in`× per source frame. **Batch size will not fix a feed-bound pipeline** ? a larger batch just means the GPU waits longer for one big batch instead of many small ones; aggregate throughput is still gated by the one CPU core assembling it. The fix is *parallelism in the feed* (A1/A2: a thread-pool feeder) and *eliminating redundant transforms* (C3), not a bigger batch and not a faster kernel.

Corollary for spend discipline (D8): do **not** provision a larger GPU. The correct next hardware lever, if CPU-bound persists after A1/A2, is **more vCPUs**, not more GPU.

---

4. Measurement protocol

Fixed control: the same L4 Cloud Run job image, the **same** ~7.5-min 60fps badminton clip, same weights, `device=cuda`. One variable per run. Record from the same instance type to keep vCPU count constant.

Primary metrics (per run):
1. **Phase-2 wall-clock (s)** ? the detector-pass timer already logged in `wasb_infer.py` (`detection done in {t}s`). This is the headline number; baseline 727.9s.
2. **GPU duty cycle (%)** ? sample `nvidia-smi --query-gpu=utilization.gpu --format=csv -l 1` (or `dmon`) across the run; report mean + p50 + fraction-of-time >50%. Baseline <1%. This tells you *whether* a change actually unblocked the GPU vs just shuffled CPU work.
3. **Secondary:** win/s from the existing progress log; peak host RSS (the bounded prefetch queue must not OOM the 32GB box); per-frame CPU time if you can `py-spy` the feeder.

A/B matrix (run in this order ? cheapest, highest-confidence first):
- Baseline: current default (`batch 8`, no prefetch, cu113). *Reconfirm 727.9s.*
- Run 1: `batch 64, prefetch 4` (the current preset) ? confirms the already-shipped opt-in and quantifies how little batch alone buys on a feed-bound run (expected: modest).
- Run 2: **A1/A2 thread-pool feeder** at `num_workers`-equivalent 4 and 8. *Expected: the big jump.*
- Run 3: **C1 torch cu121 bump**, feeder from Run 2 held constant. *Isolates the JIT/kernel win.*

- Run 4 (opt-in track): B3 NVDEC and/or B1 FP16, only if Runs 2?3 leave the GPU still underfed or now GPU-bound.

Decision rule: a change graduates to **default-on** only if (a) it strictly reduces Phase-2 wall on the control clip, AND (b) it **passes the parity check below**. A change that wins on wall-clock but fails parity graduates to **opt-in** only.

The parity check that MUST pass before any default-on change ships:
- Run the clip through the change, produce the trajectory CSV **and** the candidate windows.
- Assert against the pinned golden output from the D5/D16 parity test: **byte-identical** for the disk-frames-vs-streaming invariant, and within the runner's documented **~1e-3 px** cross-hardware tolerance for the trajectory. (This is the existing test asserted in the repo; do not hand-roll a new tolerance.)
- Additionally run **streaming vs disk-frames on the same clip** and assert they still match ? several levers here touch only the streaming path, so the two paths must not diverge.
- Run the change **twice** and assert run-to-run byte-identical output on the fixed L4 (determinism), exactly as `native_wasb_runner.py`'s acceptance gate did on the RTX 2070.

Per the pre-LGTM gate: none of these ship on the PR description's claim ? the parity test must be **run green locally/on the box**, plus `ruff` + `mypy` + full suite + coverage-floor, before LGTM.

---

5. Parity-breaking changes that MUST be opt-in (never a silent default)

Flagged explicitly ? each of these perturbs detector numerics and will fail the D5/D16 byte-parity test:

- **B1 FP16 / autocast** ? different mantissa ? different heatmap argmax ? different sub-pixel (x,y). **Opt-in.**
- **B2 `cudnn.benchmark=True` and/or TF32** ? nondeterministic algo choice + TF32 truncation both shift bytes. **Opt-in.** (Note: `cudnn.benchmark` also breaks run-to-run determinism, which the acceptance gate explicitly relies on.)
- **B3 NVDEC / GPU decode / `decord`** ? the colorspace conversion is not bit-identical to the cv2 `BGR2RGB` PNG round-trip the parity invariant is pinned to (the entire `_bgr_to_pil_rgb` "max|diff|=0" guarantee is a cv2 fact). **Opt-in.**
- **C1 torch cu113?cu121 bump** ? *should* be numerically stable but a 1.11?2.4 jump can shift cuDNN defaults; **treat as parity-must-pass-before-default.** If the gate fails, it becomes opt-in.
- **C2 GPU-side / vectorized transform** ? if the batched or CUDA ToTensor is not bit-identical to the current per-frame CPU ToTensor, it's a numeric change ? **opt-in**. (If a batched *CPU* transform proves byte-identical, it's parity-safe and can default ? verify empirically.)

Parity-SAFE (eligible for default-on after wall-clock A/B): A1/A2 thread-pool feeder, A3 batch size, and C3 transform-dedup ? all reproduce the exact same input tensors and the exact same forward; they change only *concurrency* and *redundant recomputation*, never per-frame numerics. The existing `batch_size`/`prefetch_batches` knobs are the precedent: same forward, just grouped/overlapped.

TL;DR for the DEPLOY_REPORT

1. The L4 is **CPU-feed-starved, not GPU-bound** (util <1%). Fix the feed first.
2. **Highest-value safe PR:** parallelize the streaming feed ? turn `_PrefetchIter` into an N-thread pool and lift the `num_workers=0` lock via threading (parity-safe). Add transform-dedup across overlapping windows.
3. **Bump torch 1.11+cu113 ? 2.4.1+cu121** (base image is already cuda 12.4): likely the biggest *single* discrete win by killing sm_89 PTX-JIT; ship as default *only if* the parity gate passes, else opt-in.
4. **Do NOT** reach for FP16/TF32/cudnn.benchmark/NVDEC as defaults ? every one breaks the D5/D16 byte-parity guarantee and must be an explicit opt-in that re-baselines the golden.
5. **Do NOT** buy a bigger GPU. If still CPU-bound after the feed fix, add vCPUs.

Files that change per lever: feeder/dedup ? `backend/pipeline/detectors/wasb_infer.py` (`_PrefetchIter`, the `run()` DataLoader block, streaming `num_workers=0`); new opt-in knobs ? `backend/config/models.py::WasbIndexConfig` + threaded through `backend/pipeline/detectors/native_wasb_runner.py` argv + surfaced in the `cloud-serving` preset in `backend/config/presets.py`; torch bump ? `deploy/cloudrun/Dockerfile` (lines 44?47, the `wasb` env pip install).